

GrowthBot DE

Resumo do projeto cobrindo intenção do produto, arquitetura atual, restrições de engenharia e roadmap futuro.

1. Projeto

Nome	GrowthBot DE
Tipo	Plataforma de vendas assistidas por IA com entrada conversacional via Telegram e dashboard
Mercado principal	DACH, com interações do bot em alemão e recomendações de produtos nesse contexto
Estado atual	Protótipo funcional local com orquestração conversacional, automação de follow-up, rastreamento

2. Contexto do Projeto

O GrowthBot DE foi concebido para ser mais do que um experimento de chatbot. O projeto busca validar como um assistente baseado em modelos de linguagem pode apoiar recomendação de produtos, reengajamento de usuários e monitoramento operacional dentro de um fluxo de vendas leve.

A implementação atual combina um bot no Telegram para interação com leads, uma camada de recomendação orientada por LLM, persistência local em SQLite e um dashboard em Next.js para acompanhar sessões, cliques e atividades de follow-up.

3. Objetivo Claro

O objetivo central do produto é transformar intenção conversacional em um fluxo rastreável de recomendação: detectar interesse, mapear produtos relevantes, gerar uma resposta persuasiva, armazenar a interação, agendar reengajamento e expor o resultado operacional para um administrador.

- Oferecer ao usuário uma experiência guiada no Telegram com atalhos por nicho e entrada em texto livre.
- Usar saída estruturada do LLM para recomendar exatamente um produto a partir de um catálogo curado.
- Persistir sessões e agendar follow-ups automaticamente.
- Rastrear cliques de afiliado e expor métricas centrais em uma superfície administrativa.

4. Objetivos

- Construir um assistente de vendas com IA demonstrável, simples o suficiente para rodar localmente e claro o bastante para ser explicado a recrutadores, colaboradores ou stakeholders.
- Separar orquestração conversacional de dashboard e tracking para que a arquitetura possa evoluir além do estágio de protótipo.
- Criar uma base para futura migração de persistência local para um ambiente compartilhado live.
- Manter o sistema compreensível: sem abstrações desnecessárias, com limites modulares claros e fluxo de controle direto.

5. Stacks

Frontend / Admin	Next.js 15.5.2, React 19.1.1, React DOM 19.1.1
Linguagem / Build	TypeScript 5.9.3, tsx 4.20.5
Canal do bot	node-telegram-bot-api 0.67.0
Camada LLM	OpenAI SDK 6.35.0
Validação	zod 4.1.5
Persistência	SQLite via better-sqlite3 12.9.0
Configuração	dotenv 17.4.2
Testes	Vitest 3.2.4

6. Estrutura

- src/integrations/telegram/: adaptador do Telegram responsável por mensagens, callbacks e comandos de reset de sessão.
- src/modules/conversation/: orquestrador principal que carrega histórico, classifica intenção, encontra produtos, monta prompt, chama OpenAI e persiste resultados.
- src/modules/intent/: classificação de intenção baseada em palavras-chave.
- src/modules/products/: lógica de matching e ranking de produtos com base em tags.
- src/llm/: construtor de prompt e geração estruturada de resposta via OpenAI.
- src/modules/followup/: envio agendado de follow-up com links rastreados.
- src/database/: cliente SQLite e repositórios para sessões, cliques e follow-ups.
- src/app/: rotas Next.js para dashboard, healthcheck e redirect de tracking.
- src/catalog/: catálogo estático de produtos e helpers de acesso.

7. Visão Atual

Hoje o projeto se comporta como um protótipo local forte e como um bom case de arquitetura para portfólio. O sistema já demonstra um ciclo completo entre entrada conversacional, recomendação, persistência, agendamento de follow-up, tracking de redirect e visibilidade em dashboard.

Ao mesmo tempo, a implementação ainda não pode ser considerada uma plataforma live pronta para produção. O principal limitador é o banco SQLite local, que dificulta a separação entre runtime web e runtime do bot em ambientes distintos sem ajustes de infraestrutura.

8. Pontos Fortes Atuais

- Limites modulares claros entre bot, orquestração, uso de LLM, persistência e dashboard.
- Superfície operacional demonstrável em vez de um protótipo apenas de backend.
- Geração estruturada de recomendação, e não saída livre e imprevisível do modelo.
- Fluxo de produto prático: interesse -> recomendação -> persistência -> follow-up -> tracking de clique.
- Narrativa forte para portfólio: transição de um bot simples para uma plataforma operacional com IA.

9. Restrições Atuais

- SQLite é local e hoje é o principal bloqueio para um deploy multi-serviço realmente live.
- A classificação de intenção baseada em palavras-chave pode perder nuances ou intenções mistas.
- Ainda não existe autenticação, controle de acesso, trilha de auditoria ou separação madura de ambientes para uso operacional.
- A suíte de testes existe, mas a configuração e resolução de ambiente ainda precisam de limpeza para rodar de forma confiável em qualquer contexto.
- O runtime do bot com polling é prático para uso local, mas não é o modelo final ideal para uma exposição pública madura.

10. Próximos Sprints

- Sprint 1 - base de readiness para produção: rotacionar segredos, limpar tratamento de ambiente, definir .env.example e documentar startup seguro local.

- Sprint 2 - migração para persistência compartilhada: substituir SQLite por Postgres ou Turso para permitir runtimes separados de bot e web com a mesma base.
- Sprint 3 - demo deployável: publicar o dashboard em Next.js e estabilizar um runtime do bot com persistência compartilhada.
- Sprint 4 - qualidade de recomendação: melhorar detecção de intenção, matching de catálogo, comportamento de fallback e validação estruturada da saída do LLM.
- Sprint 5 - polimento recruiter-facing: README melhor, resumo de arquitetura, screenshots e um caminho público de demo que funcione sem intervenção manual.

11. Visão Futura

A visão de longo prazo é uma plataforma de vendas com IA realmente deployável, em que entrada conversacional, recomendação, persistência, automação de follow-up, medição de cliques e insights operacionais funcionem como um sistema único e compartilhado.

- Banco remoto compartilhado sustentando todos os runtimes.
- Deploy estável de componentes web e bot.
- Melhor detecção de intenção e lógica de confiança nas recomendações.
- Analytics mais limpos sobre cliques, sucesso de follow-up e sinais de conversão.
- Possível suporte a outros canais além do Telegram, desde que a arquitetura central permaneça estável.

12. Práticas de Engenharia a Evitar

Este projeto ganha mais com disciplina do que com complexidade extra. As práticas abaixo devem ser evitadas explicitamente ao codar, criar componentes ou alterar a estrutura.

- Não introduzir abstrações antes que exista pressão real por elas. Indireção prematura tornaria um sistema hoje compreensível muito mais difícil de manter.
- Não misturar UI, tratamento do bot e lógica de orquestração no mesmo módulo. Os limites atuais são um dos ativos mais fortes do projeto.
- Não adicionar componentes ou rotas sem necessidade real do operador ou do usuário. Toda nova superfície deve servir a um fluxo de produto rastreável.
- Não acoplar novos fluxos a uma visão local-only de armazenamento. Toda mudança de dados deve considerar a futura migração para persistência compartilhada.
- Não depender de variáveis de ambiente não documentadas, setups ocultos ou conhecimento operacional apenas verbal.
- Não permitir que respostas do LLM passem sem checagem estruturada quando a aplicação espera exatamente uma recomendação válida.
- Não criar complexidade visual no dashboard sem melhorar a compreensão operacional.
- Não expor segredos em arquivos do repositório, screenshots, logs ou ambientes públicos.

13. Regras Práticas para Mudanças Futuras

- Ler o fluxo existente antes de editar: adaptador Telegram -> orquestrador -> repositórios -> dashboard/rotas de redirect.
- Preservar fluxo de dados simples. Nova lógica deve entrar onde a responsabilidade já existe, não em um arquivo aleatório por conveniência.
- Ao adicionar um componente, perguntar se ele melhora a leitura operacional ou apenas adiciona ruído visual.
- Ao mexer em armazenamento, projetar pensando em segurança de migração e fronteiras claras de repositório.
- Ao alterar comportamento do LLM, manter prompt, seleção do produto e validação de saída consistentes entre si.
- Ao adicionar testes, priorizar comportamento crítico de negócio: mapeamento de intenção, seleção de produto, efeitos de persistência e tracking de redirect.

14. Ações Imediatas Recomendadas

- Rotacionar imediatamente as credenciais expostas do Telegram e da OpenAI.
- Adicionar um README público explicando arquitetura, startup local e limitações atuais da demo.
- Preparar um caminho mínimo de demo live antes de apresentar o projeto como acessível publicamente.
- Priorizar persistência compartilhada antes de afirmar que bot e dashboard estão totalmente live juntos.

Nota final: o GrowthBot DE já é forte como case de arquitetura e engenharia de produto. O próximo passo não é aumentar superfície, e sim elevar realismo de deploy e confiabilidade operacional.